

Research Article

Enhancing Multidimensional Knapsack Problem Optimization via a Hybrid Slime Mould–Sine Cosine Algorithm

Submission ID	5675f682-d9ca-4033-93ad-6229c161be54
Manuscript ID	1241051
Submission Version	Initial Submission
PDF Generation	27 Nov 2025 22:55:24 EST by Atypon ReX

Authors

Prof. Shih-Wei Lin

 [ORCID](https://orcid.org/0000-0003-1343-0838)
<https://orcid.org/0000-0003-1343-0838>

Affiliations

- Department of Information Management, Chang Gung University, Taoyuan, Taiwan
- Department of Emergency Medicine, Keelung Chang Gung Memorial Hospital, Keelung, Taiwan
- Department of Industrial Engineering and Management, Ming Chi University of Technology, Taipei, Taiwan

Prof. Yun-Xiang Chi

Affiliations

- Department of Information Management, Chang Gung University, Taoyuan, Taiwan

Prof. Kuan-Fu Chen

Affiliations

- Department of Emergency Medicine, Keelung Chang Gung Memorial Hospital, Keelung, Taiwan
- College of Intelligent Computing, Chang Gung University, Taoyuan, Taiwan

Prof. Kuo-Ching Ying
Corresponding Author
Submitting Author

 [ORCID](https://orcid.org/0000-0002-9549-5290)
<https://orcid.org/0000-0002-9549-5290>

Affiliations

- Department of Industrial Engineering and Management, National Taipei University of Technology, Taipei, Taiwan

Files for peer review

All files submitted by the author for peer review are listed below. Files that could not be converted to PDF are indicated; reviewers are able to access them online.

Name	Type of File	Size	Page
Integrating LR with a HM for the MK_with authors.docx	Main Document - MS Word	385.9 KB	Page 4

Enhancing Multidimensional Knapsack Problem Optimization via a Hybrid Slime Mould–Sine Cosine Algorithm

Shih-Wei Lin,^{a,b,c} Yun-Xiang Chi^a, Kuan-Fu Chen^{b,d}, Kuo-Ching Ying^{e*}

^a Department of Information Management, Chang Gung University, Taoyuan, Taiwan

^b Department of Emergency Medicine, Keelung Chang Gung Memorial Hospital, Keelung, Taiwan

^c Department of Industrial Engineering and Management, Ming Chi University of Technology, Taipei, Taiwan
Email: swlin@mail.cgu.edu.tw

^a Department of Information Management, Chang Gung University, Taoyuan, Taiwan
Email: pluo2094@gmail.com

^b Department of Emergency Medicine, Keelung Chang Gung Memorial Hospital, Keelung, Taiwan

^d College of Intelligent Computing, Chang Gung University, Taoyuan, Taiwan
Email: kfchen@mail.cgu.edu.tw

^e Department of Industrial Engineering and Management, National Taipei University of Technology, Taipei, Taiwan
Email: kcying@ntut.edu.tw

Abstract

This study aims to improve the solution approach for the multidimensional knapsack problem (MKP). Two improved versions of the slime mould algorithm and the sine cosine algorithm were developed using a linear relaxation-based evaluation method to enhance solution updates. Furthermore, a hybrid slime mould–sine cosine algorithm (HSMSCA) with an advanced learning mechanism was proposed to optimize solution strategies for the MKP. The effectiveness of the algorithm was validated through experiments on four different dataset sizes, demonstrating that the proposed HSMSCA outperformed other state-of-the-art heuristic methods.

Keywords: Multidimensional knapsack problem, Hybrid algorithms; Hybrid slime mould, Sine cosine algorithm, Linear relaxation, Discrete optimization problems

* Corresponding Author. E-mail address: kcying@ntut.edu.tw (K.-C. Ying)

1. Introduction

The multidimensional knapsack problem (MKP) is a critical optimization problem with significant applications across various domains, including cargo loading [1], item selection [2], and resource allocation [3]. In these applications, the objective of the MKP is to select a subset of items that satisfy multiple resource constraints while maximizing overall profitability. Unlike the traditional knapsack problem, the MKP involves multiple resources and constraints, which increase the problem's dimensionality and complexity, making it more challenging to solve.

The MKP is defined as follows: Given a set of n items, each item i has a profit p_i and is subject to m resource constraints. The j -th resource consumed by the i -th item has a weight w_{ji} , and the upper limit for each resource j is c_j . The objective of the MKP is to select a subset of items that maximize the total profit while satisfying all resource constraints. Formally, the MKP can be expressed as follows:

$$\text{Maximize } F(X) = \sum_{i=1}^n p_i x_i \quad (1)$$

subject to

$$\sum_{i=1}^n w_{ji} x_i \leq c_j, j = 0, 1, 2, \dots, m \quad (2)$$

$$x_i = 0 \text{ or } 1, i = 0, 1, \dots, n \quad (3)$$

$$p_i > 0, w_{ji} > 0, c_j > 0 \quad (4)$$

Solutions to the MKP can be divided into two main categories: exact algorithms and heuristic algorithms. Exact algorithms include dynamic programming [4], branch-and-bound [5], backtracking algorithms [6], and enumeration algorithms [7]. Since the MKP is an NP-hard problem, these methods are only practical for small-scale problems with manageable computational times. However, as the number of items and constraints

increase, these deterministic methods become computationally inefficient.

The second category comprises heuristic algorithms, which have gained significant attention for solving the MKP. These methods primarily include approximate algorithms [8], randomized algorithms [9], and meta-heuristics. Various meta-heuristic algorithms have been explored in the literature, such as the particle swarm optimization algorithm [10], tabu search [11], the whale optimization algorithm [12] and the gray wolf optimization algorithm [13]. These approaches are considered effective for addressing the extensive computational times required by exact techniques, particularly for large-scale problems.

In recent years, numerous multi-heuristic algorithms have been developed to solve the MKP. Abdel-Basset, El-Shahat [12] introduced a modified multi-verse optimization (MMVO) algorithm to address both the knapsack problem (KP) and the MKP. MMVO utilizes a two-stage repair strategy to manage constraints and applies penalty functions to assign negative values to infeasible solutions, ensuring that their fitness values are lower than those of feasible solutions. To avoid local optima, MMVO reinitializes the population after a specified number of iterations while preserving the best solution identified. Additionally, MMVO employs a V-transfer function for code discretization.

Abdel-Basset, Mohamed [14] proposed a binary version of the slime mould algorithm (SMA) for the MKP. Since SMA was originally designed for continuous optimization problems and is not directly applicable to the discrete MKP, three adaptation functions (V-type, S-type, and U-type) were developed. These functions were integrated with the standard algorithm to create the binary slime mould algorithm (BSMA), thereby making it suitable for solving the MKP. To address the issue of local optima, Abdel-Basset, Mohamed [14] introduced two enhancement strategies: randomly flipping unselected items and reinitializing the population after a specified

number of iterations. These improvements were combined to create a more efficient variant, known as the improved binary slime mould algorithm (IBSMA). The study also incorporated a repair mechanism to handle constraints and infeasible solutions, demonstrating IBSMA's effectiveness in solving both small and large instances of MKP problems compared to other state-of-the-art algorithms. Additionally, Abdel-Basset, Mohamed [14] combined these enhancements to develop a more efficient variant of the sine cosine algorithm (SCA), known as the improved binary sine and cosine algorithm (IBSCA), specifically designed for the MKP. Feng, Wang [15] introduced a novel hybrid learning moth search (HLMS) algorithm specifically tailored for the MKP. HLMS combines global-best harmony search (GHS) with Baldwinian learning to enhance the efficiency of individual searches within the value space and leverage information from other individuals to modify the search space. GHS emphasizes global exploration, while Baldwinian learning focuses on local exploitation of solutions.

The contributions of this study include the following:

- (1) An improved slime mould algorithm (ISMA) and an improved sine cosine algorithm (ISCA) are proposed to address the MKP by employing a linear relaxation method. *The linear* relaxation of the MKP is obtained from the MILP formulation by replacing the binary decision variables x_i ($i = 0, 1 \dots n$) with continuous variables $x'_i \in [0, 1]$ ($i = 0, 1 \dots n$).
- (2) A linear relaxation-enhanced hybrid metaheuristic, named hybrid slime mould–sine cosine algorithm (HSMSCA), is proposed for MKPs.
- (3) The solutions obtained can be used as a benchmark for future comparisons with novel algorithms.

The remainder of this study is organized as follows: Section 2 reviews the fundamentals of SMA and the SCA. Section 3 details the research methodology,

including the MKP solution methods using ISMA, ISCA, and their hybrid forms. This section also covers the application of transformation functions, the generation of feasible solutions, and the implementation of ISMA, ISCA, and HSMSCA. Section 4 presents the experimental results. Section 5 summarizes the findings and provides suggests directions for future research.

2. Literature Review

The MKP is a classical challenge in combinatorial optimization and plays a critical role in numerous practical applications. This study focuses on solving the MKP using the SMA and the SCA. The SMA is inspired by the biological behaviors of slime moulds, while the SCA is based on mathematical properties. Both algorithms have gained significant prominence in the field of combinatorial optimization in recent years. The literature review of this study examines research related to the MKP, SMA, and SCA.

2.1 Slime Mould Algorithm

The SMA, introduced by Li, Chen [16], is a novel meta-heuristic algorithm inspired by the behavioral and morphological changes of slime moulds. Compared to other intelligent optimization algorithms, the SMA offers several advantages: it is based on a simple principle, requires few adjustment parameters, has strong optimization capabilities, and is easy to implement.

The SMA has been widely applied in various fields. Table 1 summarizes recent improvements to the SMA.

Table 1. A summary of recent improvements in the SMA

Algorithms	Description
SMA-AGDE [17]	An advanced hybrid approach combines the Slime Mould Algorithm (SMA) with the Adaptive Guided Differential Evolution Algorithm to address combinatorial optimization problems.
MSMA [18]	The Multi-strategy Slime Mould Algorithm (MSMA) integrates a wrap food mechanism, improves solution quality, and the interior-point method to optimize high-dimensional, nonlinear multi-reservoir hydropower systems effectively.
FP-SMA [19]	The FP-SMA adaptively adjusts the population size to enhance the SMA's efficiency, effectively balancing exploration and exploitation.
CO-SMA [20]	CO-SMA combines chaotic search and crossover-opposition strategies to enhance the slime mould algorithm's exploration and exploitation.
ESMA [21]	The hybrid algorithm combines sine cosine updates with a modified sigmoid function to enhance the slime mould algorithm's convergence and exploration.
SMAFA [22]	SMAFA enhances feature selection by integrating the exploration strengths of the firefly algorithm with those of the slime mould algorithm.

The mathematical concept of the SMA can be described as follows.

Slime moulds approach food by detecting airborne odors, and their approximation behavior is represented by Equation (5), where $\vec{vb}$ is determined by Equations (6) and (7).

$$\vec{S}(t+1) = \begin{cases} \vec{S}_b(t) + \vec{vb}(W \times \vec{S}_A(t) - \vec{S}_B(t)), & r < p \\ \vec{vc} \times \vec{S}(t), & r \geq p \end{cases} \quad (5)$$

$$\vec{vb} = [-a, a] \quad (6)$$

$$a = \operatorname{arctanh} \left(-\left(\frac{t}{t_{\max}} \right) + 1 \right) \quad (7)$$

Here, the vector $\vec{vb}$ ranges from $-a$ to a . The vector $\vec{vc}$ linearly decreases from 1 to 0. t represents the current iteration, and $t_{\max}$ is the maximum number of iterations. r is a random variable between 0 and 1. $\vec{S}_b$ is the solution obtained in the previous iteration, and $\vec{S}(t+1)$ is the newly generated candidate solution. $\vec{S}(t)$ is the position of the current individual, while $\vec{S}_A$ and $\vec{S}_B$ are two distinct individuals selected from the population. p in Equation (5) is computed using Equation (8).

$$p = \tanh(|F(i) - DF|) \quad (8)$$

where $i \in 1, 2, 3, \dots, N$, $F(i)$ is the fitness value of current individual i , and DF is the fitness value of the best solution in the previous iteration ($\vec{S}_b$). $\vec{W}$ is the weight of slime moulds, calculated using Equations (9) and (10).

$$\vec{W}(\text{smellindex}(l)) = \begin{cases} 1 + r \times \log\left(\frac{bF - F(i)}{bF - wF} + 1\right), & \text{condition} \\ 1 - r \times \log\left(\frac{bF - F(i)}{bF - wF} + 1\right), & \text{other} \end{cases} \quad (9)$$

$$\text{smellindex} = \text{sort}(F) \quad (10)$$

where r is a random value in the interval $[0, 1]$, bF is the best fitness value in the previous generation, wF is the worst fitness value in the previous generation, smellindex denotes the sequence of sorted fitness values (in descending order for a maximum value problem), “condition” represents the top 50% of the population.

To simulate the contraction pattern in a slime mould's venous tissue structure, the behavior of food encapsulation can be modeled mathematically. When considering food quality, the slime mould generates a new solution or location, assigning greater weight to regions with higher food concentrations. This shift in weight prompts the exploration of other regions. In scenarios where food concentration is high, slime moulds must decide whether to leave the current region to search for different food sources or remain in the current region. This decision prevents them from becoming confined to the best available source and promotes the search for multiple food sources across different regions.

The slime mould's position can be expressed using Eq. (11), reflecting its behavior in seeking multiple food sources.

$$\vec{S}(t+1) = \begin{cases} \text{rand} \times (UB - LB) + LB, & r_1 < z \\ \vec{S}_b(t) + \vec{vb} \times (\vec{W} \times \vec{S}_A(t) - \vec{S}_B(t)), & r_2 < p \\ \vec{vc} \times \vec{S}(t), & r \geq p \end{cases} \quad (11)$$

where r_1 and r_2 are two random values between 0 and 1, and UB and LB represent the upper and lower bounds of the search space, respectively. z is the parameter that determines whether the current individual seeks another food source to avoid falling

into a localized trap or is used to find a better solution within the current search space.

The SMA starts with a set of randomized initial solutions and records the best solution found so far as the current best. Simultaneously, the algorithm updates other solutions to identify the current best solution. As iterations progress, the algorithm adjusts the range of parameters $\vec{vb}$ and $\vec{vc}$ to improve the exploration of the search space. The optimization process concludes when the number of iterations reaches the predefined maximum. The pseudocode for the SMA is presented in Figure 1.

Algorithm1: Pseudocode of Slime Mould Algorithm	
1	<i>Initialize the parameters pop_size, Max_iteration, a</i>
2	<i>Initialize the positions of slime mould $SM^l(l = 1, 2, \dots, n)$</i>
3	While ($t \leq \text{Max_iteration}$)
4	<i>Calculate the fitness of all slime mould;</i>
5	<i>Update best fitness F and the current best solution x_b</i>
6	<i>Calculate the $\vec{W}$ by Eq. (9)</i>
7	For each search portion
8	<i>Update $p, \vec{vc}, \vec{vb}$</i>
9	<i>update positions by Eq. (11)</i>
10	End For
11	$t = t + 1$
12	End while
13	Return the best solution x_b

Figure 1. The Pseudocode of SMA

2.2 Sine Cosine Algorithm

Mirjalili [23] introduced the SCA, a population-based optimization algorithm that generates multiple stochastic candidate solutions based on the mathematical properties of the sine and cosine functions. The SCA leverages the variations in the sine and cosine function values to guide the optimization process. The algorithm performs global exploration when the sine or cosine values are greater than 1 or less than -1. Conversely, it focuses on local exploitation when the sine or cosine values lie between -1 and 1.

The SCA integrates several stochastic and adaptive variables to balance exploration and exploitation throughout optimization. It has been extensively applied across various fields. Table 1 summarizes recent improvements in the SCA.

Table 2. A summary of the recent improvements in the SCA

Algorithm	Description
MSCA [24]	MSCA enhances the Sine Cosine algorithm with Lévy flights and adaptive search agents for efficient OPF problem-solving.
mSCA [25]	The m-SCA enhances the Sine Cosine Algorithm by incorporating oppositional populations and self-adaptive components to improve global optimization.
MSCA-ELM [26]	The MSCA-ELM combines a modified Sine Cosine Algorithm with an Extreme Learning Machine for efficient pathological brain MRI classification.
DDSCA [27]	DDSCA enhances the Sine Cosine Algorithm by using dimension-by-dimension evaluation and dynamic control for global optimization.

In the SCA, the positions of m individuals are randomly generated, with each solution corresponding to an individual's position in the search space. The position of the i -th individual is denoted as $X_i = (X_{i1}, X_{i2}, \dots, X_{in})^T$, where n represents the dimension of the individual. The best position experienced by all current individuals is denoted as $P_i = (P_{b1}, P_{b2}, \dots, P_{bn})^T$. In the subsequent iteration, the position update for the i -th individual is governed by either Equations (12) or (13).

$$X_i^{t+1} = X_i^t + r_1 \times \sin(r_2) \times |(r_3 P_i^t - X_i^t)| \quad (12)$$

$$X_i^{t+1} = X_i^t + r_1 \times \cos(r_2) \times |(r_3 P_i^t - X_i^t)| \quad (13)$$

where X_i^t denotes the position of the solution in the i^{th} dimension at the t^{th} iteration, and P_i represents the position of the target solution in the i^{th} dimension. r_2 and r_3 are random values within the ranges $[0, 2\pi]$ and $[0, 2]$, respectively, while $|\cdot|$ denotes the absolute value.

To combine Equations (12) and (13) and incorporate the random variable r_4 (where r_4 is a random value in the range $[0, 1]$), Equation (14) is derived:

$$X_i^{t+1} = \begin{cases} X_i^t + r_1 \times \sin(r_2) \times |(r_3 P_i^t - X_i^t)| & r_4 < 0.5 \\ X_i^t + r_1 \times \cos(r_2) \times |(r_3 P_i^t - X_i^t)| & r_4 \geq 0.5 \end{cases} \quad (14)$$

Equation (14) indicates that the SCA has four main variables: r_1, r_2, r_3 , and r_4 .

The variable r_1 determines whether the next position of the solution is within the space between the current solution and the target or outside of it. By adapting r_1 , the algorithm balances both exploration and exploitation.

$$r_1 = a \left(1 - \frac{t}{T}\right) \quad (15)$$

The r_1 variable controls for both stages in SCA is provided by Equation (15), where t is the current iteration number, T is the maximum iteration number, and a is a constant parameter in the SCA. r_2 is a random value between $[0, 2\pi]$, which determines the movement step in the next iteration. r_3 is a random value between $[0, 2]$ and r_4 is a random value between $[0, 1]$, facilitating the transition between sine and cosine functions according to Equation (14). When $r_4 < 0.5$, the position is updated using Equation (12); when $r_4 \geq 0.5$, Equation (13) is used.

Since Mirjalili [23] utilized sine and cosine functions in Equations (12) and (13), the algorithm is named the SCA. The impact of these functions on updating Equation (14) is illustrated in Figure 2, which demonstrates the search area of the solution with various value ranges. Figure 2 highlights that Equation (14) defines two distinct search directions within the search space.

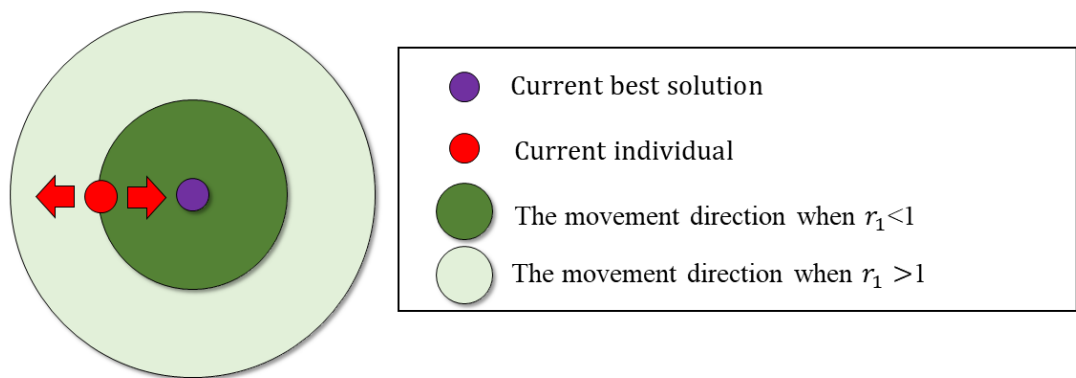


Figure 2. Effect of different values of r_1 on the solution's search direction

Although Figure 2 depicts a two-dimensional model, Equation (14) can be

extended to higher dimensions. The periodic properties of the sine and cosine functions facilitate the iteration between positions, ensuring that the positions generated during the exploitation phase remain within defined bounds. To explore a broader solution space, the subsequent position should extend beyond these bounds, which can be achieved by adjusting the amplitudes of the sine and cosine functions.

Figure 3 illustrates the influence of sine and cosine functions within the range $[-2, 2]$. It shows that the next position update is determined by adjusting the amplitudes of these functions to choose between or beyond the two positions. This selection process is guided by a stochastic variable r_2 within the range $[0, 2\pi]$. This method ensures distinct exploration and exploitation of the search space. Specifically, when the amplitudes of the sine and cosine functions fall within $[-2, -1]$ and $[1, 2]$, exploration occurs. Conversely, when the amplitudes lie within $[-1, 1]$, exploitation takes place.

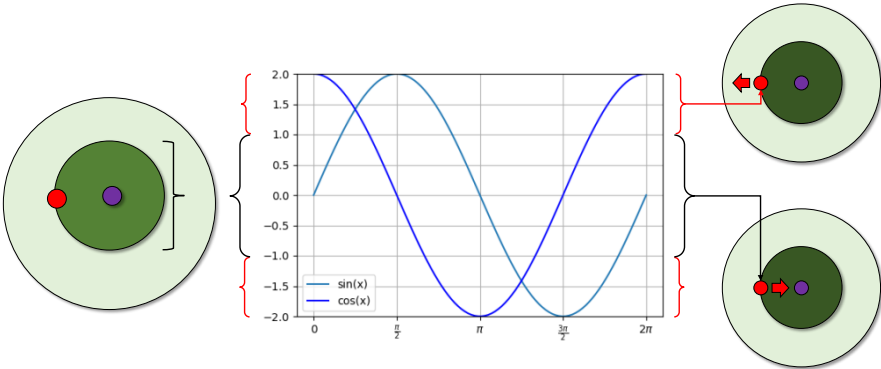


Figure 3. The effects of sine and cosine functions

Figure 4 presents the pseudocode for the SCA. The algorithm begins by initializing the parameters and generating a set of random solutions. It then evaluates these solutions using the objective function, recording the best solution found as the target point. Concurrently, the range of the sine and cosine functions are adjusted to enhance the exploitation of the search space as the number of iterations increases. The SCA algorithm concludes the optimization process when the number of iterations

exceeds the maximum iterations allowed.

Algorithm2: Pseudocode of Sine Cosine Algorithm

```

1  Initialize the parameters  $pop\_size$ ,  $Max\_iteration$ , and  $a$ 
2  Initialize the search agents
3  Evaluate each search agent using the objective function
4  Select the best solution  $x_b$  from the set of search agents
5  Initialize the iteration count  $t=0$  and variable  $r_1$ 
6  While ( $t \leq Max\_iteration$ )
7      For each search agent
8          Update the position of search agent using equations(14)
9          Evaluate the updated search agent
10         Update the best solution  $x_b$ 
11         Update the variable  $r_1, r_2, r_3$  and  $r_4$ 
12     End For
13      $t = t + 1$ ;
14 End while
15 Return the best solution  $x_b$ 

```

Figure 4. The Pseudocode of the SCA

3. The Proposed ISMA, ISCA, and HSMSCA

The SMA and SCA were originally designed for continuous variable optimization problems. However, the MKP is a binary combinatorial optimization problem. To address this, improved versions of the SMA and SCA, named the ISMA and ISCA, have been developed specifically to solve the MKP. Furthermore, a new hybrid approach, the hybrid improved slime mould and improved sine cosine algorithm (HSMSCA), has been proposed.

First, the solution representation method (3.1) and the population initialization method (3.2) are described. Next, the method for generating feasible solutions (3.3) and the transformation function (3.4) are presented. Subsequently, the repair algorithm (3.5) is examined to address infeasible solutions. The improvement algorithm (3.6) is employed to enhance the quality of the solution. Finally, the improved slime mould algorithm (ISMA) (3.7) and the improved sine cosine algorithm (ISCA) (3.8) are introduced, followed by an exploration of a hybrid application of the ISMA and ISCA

(3.9).

3.1 Solution Representation method

This study employs two types of solution codes. Since the SMA and SCA are designed for continuous variables, the nominal solution representation is a vector with n elements, where the value of each element is between 0 and 1. The actual solution representation is a binary vector with n elements, each indicating whether the corresponding item is selected in the knapsack. To convert from a nominal solution representation to an actual solution representation, a conversion function is utilized. Due to the potential infeasibility of the actual solution representation, repair and improvement algorithms are necessary to generate a feasible solution.

3.2 Population Initialization Method

This study utilized information from the MKP to develop a method for generating an initial elite population. In MKP, the price-performance ratio of an item is considered crucial because each item utilizes multiple resources to generate profit. Therefore, assessing the price-performance of items is essential before identifying the optimal subset of items to maximize the total profit. For the classic knapsack problem, the price-performance ratio can be easily measured by the profit generated relative to the resources consumed.

An effective initial population can enhance the algorithm's efficiency; however, generating an initial elite population with diversity depends on the specific problem being addressed. The price-performance ratio of an item is calculated using Equation (16).

$$u_i = -\frac{p_i}{\sum_{j=1}^m w_{ji}x'_{ji}} \quad (16)$$

where u_i represents the price-performance ratio of item i ; p_i represents the profit of item i ; w_{ij} denotes the amount of the i -th resource used by the j -th item; and x'_i represents the agent multiplier of the i -th item, which can be viewed as the relaxation solution of the i -th resource constraint in the linear relaxation of MKP. The linear relaxation of MKP is obtained by solving the MILP of MKP using continuous decision variables ($x'_i = 0 \sim 1$) instead of binary decision variables.

Figure 5 illustrates the method for generating the initial elite population. To obtain s solutions with higher profits, items with higher price-performance ratios should be prioritized. Therefore, a diverse initial elite population can be generated by randomly selecting items with higher price-performance ratios. All items $U = \{i_1, i_2, \dots, i_n | u_1 \geq u_2 \geq \dots \geq u_n\}$ should be sorted in descending order of their price-performance ratio. The algorithm selects items based on this descending order. To determine whether to include an item, the algorithm generates a random value. If this random value is less than 0.5 and satisfies the resource constraints, the decision variable representing the item is set to 1, and the remaining weight in the current dimension is updated. If the random value is greater than or equal to 0.5 or does not meet the resource constraints, the decision variable representing the item is set to 0, and the remaining capacity remains unchanged.

Algorithm3: Initial elite population generator

```

1  Initialize the search agents
2   $x_j^k = 0, k = 1, 2, \dots, s; j = 1, 2, \dots, n$ , and the fitness:
3   $f^k = 0$ 
4   $r_i = 0, \forall i = 1, 2, \dots, m$ .
5  for  $k = 1$  to  $s$  do
6    for  $i = 1$  to  $n$  do
7       $\hat{i} = i$ -th item in  $U = \{i_1, i_2, \dots, i_n | u_1 \geq u_2 \geq \dots \geq u_n\}$ 
8      if  $rand < 0.5$  and  $r_j \leq c_j, j = 0, 1, 2, \dots, m$ 
9        Then
10          $x_i^k = 1, f^k \leftarrow f^k + c_{\hat{i}}, r_j \leftarrow r_j + a_{ij}, \forall j = 1, 2, \dots, m$ .
11      End
12    End
13  End

```

Figure 5. The pseudocode of generating the initial elite population

3.3 The Feasible Solution Generation Method

To ensure that each individual's solution is binary-valued for computing the objective function value, the Tanh transformation function (Section 3.4) was employed to convert continuous values in the nominal solution representation into binary variables of 0 or 1. The repair algorithm (Section 3.5) ensures that the current candidate solution satisfies all constraints and is feasible. Finally, the improvement method (Section 3.6) was applied to enhance the quality of the current candidate solution.

3.4 Transformation Function

Transformation functions are essential for converting continuous solutions into binary ones. Since the SMA and SCA are designed for continuous optimization problems, their search strategies are ineffective for discrete or binary problems. To address this problem, a V-transformation function is introduced, which maps continuous values between 0 and 1 to binary values. Equation (17) standardizes the results by converting them to binary values of 1 or 0.

$$\mathbf{z}^k = \begin{cases} 1, & \text{if } |\tanh(\vec{S}_i)| \geq rand() \\ 0, & \text{otherwise} \end{cases} \quad (17)$$

3.5 Repair Algorithm

Because the binary solution (actual solution representation) $\mathbf{z}^k = (z_1^k, z_2^k, \dots, z_n^k)$ is derived from the nominal solution representation using Equation (17), some solutions may be infeasible. To ensure feasibility, each infeasible solution is converted into a feasible one using the repair algorithm, which sequentially removes items $U = \{i_1, i_2, \dots, i_n | u_1 \geq u_2 \geq \dots \geq u_n\}$ from the knapsack according to the descending order of their price-performance ratios until a feasible solution is achieved.

The repair algorithm proposed by Chu and Beasley [28] has been employed in various heuristic methods [29-31] to solve the MKP. This repair algorithm effectively utilizes the price-performance ratio information in the MKP to handle infeasibility. Figure 6 illustrates the pseudocode of the repair algorithm.

Algorithm4: Repair Algorithm (RA)	
1	Input: infeasible solution S_i
2	While $\left(\sum_{i=1}^n w_{ji}x_i > c_j\right)$
3	Remove the items with the lowest u_i
4	End while
5	Return S_i

Figure 6. The pseudocode of the repair algorithm

3.6 Improvement Algorithm

The improved algorithm proposed by Luo and Zhao [13] enhanced the current feasible solution, as depicted in Figure 7. The primary step involves sorting items in descending order according to their price-performance ratios $U = \{i_1, i_2, \dots, i_n | u_1 \geq u_2 \geq \dots \geq u_n\}$. Provided that all the constraints are satisfied, the items with the highest price-performance ratios are sequentially added to the knapsack.

Algorithm5: Improvement Algorithm (IA)

```

1  Input: Feasible solution  $S_i$ 
2  for each items
3      If  $\left(\sum_{i=1}^n w_{ji}x_i \leq c_j\right)$ 
4          Then
5              Insert the items with the height  $u_i$ 
6  End while
7  Return  $S_i$ 

```

Figure 7. The pseudocode of the improvement algorithm

3.7 Improved Slime Mould Algorithm

Figure 8 presents the pseudocode for the ISMA, which is an enhanced version of the original SMA algorithm. The ISMA first generates an initial elite population by evaluating the price-performance ratio of each item through a linear relaxation. During each iteration, the algorithm converts infeasible solutions into feasible ones through a repair and refinement strategy, ensuring that the solutions can effectively solve the MKP by the end of each update.

Algorithm 6: Pseudocode of Improved Slime Mould Algorithm

```

1  Initialize the parameters pop_size, Max_iteration,  $\alpha$ ;
2  Initialize the positions of each slime mould  $SM^I(I = 1,2,...,n)$ 
3  While( $t \leq \text{Max\_iteration}$ )
4      Calculate the fitness of all slime mould
5      Update best solution  $x_b$ 
6      Calculate the  $\vec{W}$  by Eq.(9)
7      For each search portion
8          Update  $p, \vec{vc}, \vec{vb}$ 
9          update positions by Eq.(11)
10         Apply a transformation function on positions  $\vec{S}_i$ 
11     End For
12     Apply Algorithm 4(RA), and Algorithm 5(IA) on the Binary of  $\vec{S}_i$ 
13      $t = t + 1$ 
14 End while
15 Return the best solution  $x_b$ 

```

Figure 8. The pseudocode of ISMA

3.8 Improved Sine Cosine Algorithm

Figure 9 shows the pseudocode for the ISCA, which is an advanced version of the SCA. The ISCA begins by using linear relaxation to evaluate the price-performance

ratio of each item, thereby generating an initial high-quality population. It then converts infeasible solutions into feasible ones using a repair-and-improvement strategy, which is applied at the end of individual updates in every iteration.

Algorithm7: Pseudocode of Improved Sine Cosine Algorithm

1. *Initialize the search agents*
 2. *Evaluate each search agent using the objective function*
 3. *Select the best solution x_b from the set of search agents*
 4. *Initialize the iteration count $t=0$ and random parameter r_1*
 5. **While**($t \leq \text{Max_iteration}$)
 6. *For each search agent*
 7. *Update the position of search agent using Eq.(14)*
 8. *Evaluate the updated search agent*
 9. *Update the best solution x_b*
 10. *Update the variables r_1, r_2 and r_3*
 11. *Apply a transfer function on positions $\vec{S}_i$*
 12. **End For**
 13. *Apply Algorithm 4(RA), and Algorithm 5(LA) on the Binary of $\vec{S}_i$*
 14. $t = t + 1;$
 15. **End while**
 16. **Return** the best solution x_b
-

Figure 9. The pseudocode of ISCA

3.9 Hybrid Slime Mould–Sine Cosine Algorithm

Figure 10 illustrates the main steps of the HSMSCA, along with its pseudocode which is shown in Figure 11. Initially, all the necessary parameters are set, and the population is initialized to represent the candidate solutions. Each solution is evaluated using a fitness function to determine its quality. Based on the fitness values, a selection mechanism determines whether to update each individual's position using the SMA or the SCA. The decision is guided by the quality of the fitness values, with the corresponding probability value defined in Equations (18) and (19). These equations ensure that the ISMA and ISCA function competitively, thereby enhancing the balance between exploration and exploitation during the search process.

$$fit_{n^{A_q}}^{A_q} = \frac{n^{A_q} - 1}{n^{A_q}} \cdot fit_{n^{A_q}-1}^{A_q} + \frac{1}{n^{A_q}} \cdot [P(\vec{S}_i)^N - P(\vec{S}_i)] / P(\vec{S}_i) \quad (18)$$

$$Prob = fit_{n^{A_1}}^{A_1} / (fit_{n^{A_1}}^{A_1} + fit_{n^{A_2}}^{A_2}) \quad (19)$$

where n^{A_q} denotes the number of times the algorithm A_q has been chosen to update the individual's position; A_1 and A_2 represent the ISMA and ISCA, respectively; $fit_{n^{A_q}}^{A_q}$ denotes the fitness value of the algorithm A_q at n^{A_q} ; $P(\vec{S}_i)$ is the objective function value of the current position; and $P(\vec{S}_i)^N$ is the objective function value of the new position.

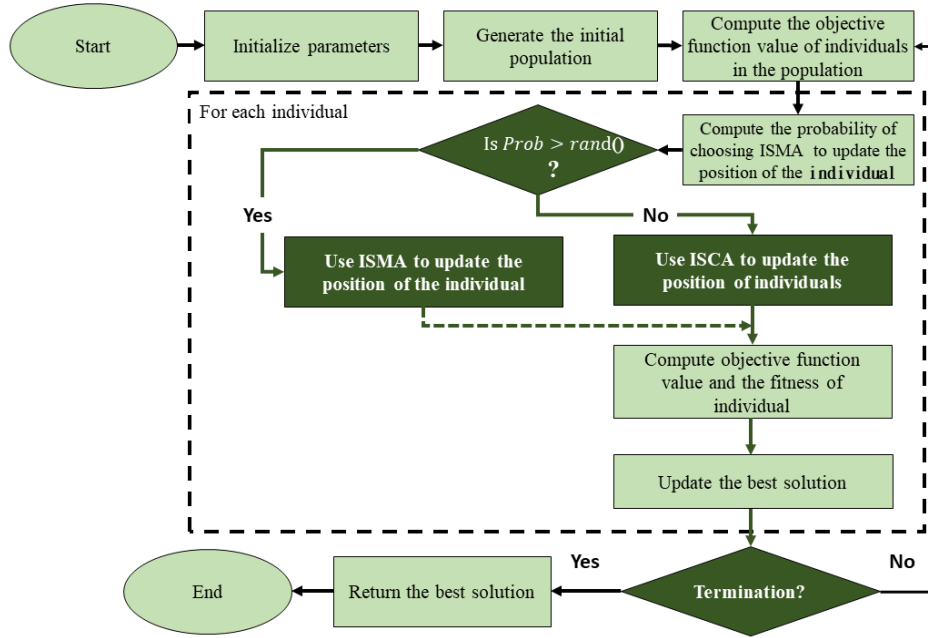


Figure 10. The flowchart of the HSMSCA algorithm

Algorithm 8: Pseudocode of Hybrid Slime Mould–Sine Cosine Algorithm

```

1  Initialize the parameters pop_size, Max_iteration, , a, z
2  Initialize the positions of each individual  $SM^I(I = 1, 2, \dots, n)$ 
3  While( $t \leq \text{Max\_iteration}$ )
4      update SMA parameter
5      update SCA parameter
6      For each individual ( $\vec{S}_i$ )
7          Choice update method using Eqs. (18) and (19)
8      End For
9      Apply Algorithm 4 (RA), and Algorithm 5 (IA) on the Binary of  $\vec{S}_i$ 
10      $t = t + 1$ 
11     Update the best solution  $x_b$ 
12 End while
13 Return the Best Solution  $x_b$ 
  
```

Figure 11. The pseudocode of the HSMSCA

4. Computational Results

In this section, several experiments were conducted to evaluate the performance of the proposed algorithm. Additionally, the proposed algorithm's effectiveness was compared with that of other competing algorithms.

4.1 The Development Environment of The Proposed Algorithms

The experiments were conducted on a Windows 11 computer with 32 GB of RAM and an Intel Xeon E5 1650v3@ 3.50 GHz processor. Each proposed algorithm was executed 20 times on the entire problem library. The algorithms were implemented using Python 3.11 in the Visual Studio Code environment.

4.2 Problem Sets Used

The performance of the algorithms was assessed using four classic MKP libraries, which are described in the literature and are available through the OR-Library (<http://people.brunel.ac.uk/mastjjb/jeb/orlib/mknapinfo.html>).

Problem Set 1 includes 16 instances with $m = 2$ to 30 and $n = 20$ to 105. It comprises problem instances from Sento [32], HP [33], PB [33], and Weing [34], which containing 2, 2, 6, and 8 instances, respectively.

Problem Set 2 contains 30 medium-sized questions with $m = 5$ and $n = 30$ to 90. These instances are labeled as Weish [35].

Problem Set 3 includes 30 large-scale problem instances divided into two subsets. Subset 3-1 consists of 15 instances with $m = 15$ and $n \in \{100, 250, 500\}$, labeled cb1, cb2, and cb3, respectively. Subset 3-2 also consists of 15 instances with $m = 15$ and $n \in \{100, 250, 500\}$, labeled cb4, cb5, and cb6, respectively [28].

Problem Set 4 contains nine questions with $m = \{15, 25, 50\}$ and $n \in \{100, 200, 500, 1000, 1500\}$, which were created by Glover and Kochenberger [36] and labeled

GK.

Table 3 provides a detailed description of the Sento, HP, PB, Weing, WEISH, CB, and GK problem instances. The "BKS" column lists the best solutions [15].

Table 3. The scales of problem sets

Set	Instance	BKS	N	M	Instance	BKS	N	M	Instance	BKS	N	M
1	Sento 1	7772	60	30	PB4	95168	29	2	Weing3	95677	28	2
	Sento 2	8722	60	30	PB5	2139	20	10	Weing4	119337	28	2
	HP1	3418	28	4	PB6	776	40	30	Weing5	98796	28	2
	HP2	3186	35	4	PB7	1035	37	30	Weing6	130623	28	2
	PB1	3090	27	4	Weing1	141278	28	30	Weing7	1095445	105	2
	PB2	3186	34	4	Weing2	130883	28	2	Weing8	624319	105	2
Set	Instance	BKS	N	M	Instance	BKS	N	M	Instance	BKS	N	M
2	weish01	4554	30	5	weish11	5643	50	5	weish21	9074	70	5
	weish02	4536	30	5	weish12	6339	50	5	weish22	3947	70	5
	weish03	4115	30	5	weish13	6159	50	5	weish23	8344	80	5
	weish04	4561	30	5	weish14	6954	60	5	Weish24	10,220	80	5
	weish05	4514	30	5	weish15	7436	60	5	Weish25	9939	80	5
	weish06	5557	40	5	weish16	7289	60	5	Weish26	9584	90	5
	weish07	5567	40	5	weish17	8633	60	5	weish27	9819	90	5
	weish08	5605	40	5	weish18	9580	70	5	weish28	9492	90	5
	weish09	5246	40	5	weish19	7698	70	5	weish29	9410	90	5
	weish10	6339	50	5	weish20	9450	70	5	weish30	11,191	90	5
Set	Instance	BKS	N	M	Instance	BKS	N	M	Instance	BKS	N	M
3	cb1-1	24381	100	5	cb3-1	120148	500	5	cb5-1	59187	250	10
	cb1-2	24274	100	5	cb3-2	117879	500	5	cb5-2	58781	250	10
	cb1-3	23551	100	5	cb3-3	121131	500	5	cb5-3	58097	250	10
	cb1-4	23534	100	5	cb3-4	120804	500	5	cb5-4	61000	250	10
	cb1-5	23991	100	5	cb3-5	122319	500	5	cb5-5	58092	250	10
	cb 2-1	59312	250	5	cb4-1	23064	100	10	bb6-1	117821	500	10
	cb 2-2	61472	250	5	cb4-2	22801	100	10	cb6-2	119249	500	10
	cb 2-3	62130	250	5	cb4-3	22131	100	10	cb6-3	119215	500	10
	cb 2-4	59463	250	5	cb4-4	22772	100	10	cb6-4	118829	500	10
	cb 2-5	58951	250	5	cb4-5	22751	100	10	cb6-5	116530	500	10
Set	Instance	BKS	N	M	Instance	BKS	N	M	Instance	BKS	N	M
4	GK-1	3766	100	15	GK-4	5767	150	50	GK-7	19221	500	25
	GK-2	3958	100	25	GK-5	7560	200	25	GK-8	18806	500	50
	GK-3	5656	150	25	GK-6	7677	200	50	GK-9	58089	1500	25

4.3 Comparison Algorithms and Evaluation Criteria

In this study, several recent MKP algorithms from the literature were selected as competing algorithms, including the hybrid learning moth search (HLMS) [15], SCA [37], SMA [14], whale optimization algorithm (WOA) [12], and modified multi-verse optimization (MMVO) [12].

The results of the comparison algorithms were compiled from the relevant literature. If the result of an algorithm for a particular MKP problem instance was not available, that problem instance was excluded from the comparison. Given that the comparison algorithms were implemented in various programming languages, executed

on different computing platforms, and configured with distinct termination conditions and parameters, the average percentage deviation ($PDev$) calculated using Equation (20) was used for comparison.

$$PDev = \frac{BKS - Mean}{BKS} * 100\% \quad (20)$$

Three statistical indicators were employed to evaluate the performance of the proposed algorithm against the competitors' algorithms: the best value, the mean value, and the percentage deviation. $PDev$ is a statistical measure that has been designed to assess the error percentage between the best-known solution (BKS) and the mean of the obtained solutions. The algorithm with the lowest average $PDev$ value is regarded as the most effective.

4.4 Parameter Calibration

This section examines the effects of various parameters on algorithm performance. First, two problem instances were randomly selected from each problem set for validation, and the experimental results were summarized in Table 4. The population was set to 20, and the termination condition was specified as 100,000 evaluations [15]. Therefore, the maximum number of generations ($Max_iteration$) was set to 5,000 (100,000/20).

The average $PDev$ of each problem instance was calculated to evaluate the performance of the parameters. Secondly, parameter combinations were tested for the ISMA with $z = \{0.01, 0.08, 0.15\}$, ISCA with $a = \{1.5, 2.0, 2.5\}$, and HSMSCA with combinations of $z = \{0.01, 0.08, 0.15\}$ and $a = \{1.5, 2.0, 2.5\}$. The evaluation results for these algorithms with different parameter combinations across the randomly selected problem instances are presented in Appendix A, showing the best, average, and $PDev$

values for all parameters. The analysis concluded that the ISMA algorithm performed optimally with $z = 0.08$, the ISCA with $a = 1.5$, and the HSMSCA with $z = 0.08$ and $a = 2.5$. As shown in Appendix A, the parameter a in ISCA has the largest impact on convergence speed, with deviations of ± 0.5 from the optimal value (1.5) causing up to a 12% increase in average PDev. Similarly, the parameter z in ISMA critically affects exploration: values below 0.05 reduce solution diversity, while values above 0.10 delay convergence by over 8%. The interaction between z and a in HSMSCA also proves essential; maintaining z in $[0.08, 0.10]$ and a in $[2.0, 2.5]$ balances exploration and exploitation most effectively. These conclusions were drawn by comparing the mean PDev values for all randomly selected problem instances.

Table 4. The result of parameter experiments

Algorithm	ISMA			ISCA					
Parameter value	$z = 0.01$	$z = 0.08$	$z = 0.15$	$a = 1.5$	$a = 2.0$	$a = 2.5$	-	-	-
Average PDev	0.072	0.065	0.073	0.066	0.094	0.114	-	-	-
Algorithm	HSMSCA								
Parameter values	$z = 0.01, a = 1.5$	$z = 0.01, a = 2.0$	$z = 0.01, a = 2.5$	$z = 0.08, a = 1.5$	$z = 0.08, a = 2.0$	$z = 0.08, a = 2.5$	$z = 0.15, a = 1.5$	$z = 0.15, a = 2.0$	$z = 0.15, a = 2.5$
Average PDev	0.081	0.068	0.067	0.066	0.064	0.063	0.067	0.065	0.071

*Bold value denotes the best value for the algorithm.

4.5 The Final Results

This section compares the optimal parameter values identified in Section 4.4 ($z = 0.01$ for ISMA, $a = 2.0$ for ISCA, and $z = 0.15$ and $a = 2.0$ for HSMSCA) with those of other state-of-the-art algorithms. As previously mentioned, all algorithms were executed 20 times on the complete set of problem instances, with detailed performance results provided in Appendix B.

The summarized results are presented in Table 5. The ISMA, ISCA, and HSMSCA algorithms demonstrate significant advantages over the other algorithms evaluated. Furthermore, the proposed HSMSCA algorithm outperforms the others,

including the ISMA and ISCA algorithms. Table 5 lists the mean *PDev* values, with the best results highlighted in bold.

The validation and comparison of the proposed algorithms were conducted on 18 instances from Problem Set 1, with specific results detailed in Appendix B. The comparison metrics include the maximum value, the mean value, and the standard deviation percentage.

The proposed algorithm was compared with five other algorithms (HLMS, IBSMA_U1, SCA, BIWOA, BMMVO) to evaluate Problem Set 2. The detailed performance data is shown in Table 5, where HSMSCA achieves the best solution across all 30 problems.

For Problem Set 3, Table 5 summarizes the corresponding experimental results. When comparison with two other algorithms (HLMS, BIWOA) in Problem Set 3-1 and with HLMS in Problem Set 3-2, the data shows that HSMSCA slightly outperforms ISMA and ISCA and significantly outperforms other algorithms in both subsets.

For Problem Set 4, the proposed algorithms were similarly validated and compared, with the results ranked according to Max/Mean/*PDev*/Rank. Table 5 again lists the average *PDev* and Rank values, with the best results marked in bold.

Table 5. The comparison results of various algorithms

Set	HSMSCA	ISMA	ISCA	HLMS	BIWOA	BMMVO	BSCA	IBSMA_U1
1	0.020/1.167	0.023/ 1.167	0.200/2.444	0.569/3.556	-/-	-/-	-/-	-/-
2	0.000/1.000	0.000/1.000	0.050/2.433	0.154/4.367	0.472/4.033	0.861/5.133	0.314/3.933	0.105/2.467
3-1	0.091/1.800	0.097/2.533	0.096/1.867	0.925/4.267	1.234/4.533	-/-	-/-	-/-
3-2	0.219/1.800	0.225/2.333	0.233/1.933	1.349/3.933	-/-	-/-	-/-	-/-
4	0.180/1.778	0.182/2.222	0.188/2.000	0.922/5.444	0.857/5.333	2.438/8.000	1.997/7.000	0.624/4.111

*Bold value denotes the best *PDev*/Rank.

- Means N/A.

To further validate the efficacy of the HSMSCA algorithm, we conducted a series of Wilcoxon signed-rank tests. At a confidence level of $\alpha = 0.05$, the statistical results presented in Table 6 clearly demonstrated that the HSMSCA algorithm exhibited

significant superiority over other algorithms, including ISMA and ISCA.

Table 6. The result of Wilcoxon sign rank on *PDev*

HSMSCA vs		ISMA	ISCA	HLMS	BIWOA	BMMVO	BSCA	IBSMA_U1
Set 1	W value	79.000	22.500	0.500	-	-	-	-
	p-value	0.766	0.005	0.000	-	-	-	-
Set 2	W value	232.500	76.500	5.000	22.500	18.000	27.500	76.500
	p-value	1.0	0.001	0.000	0.000	0.000	0.000	0.0010
Set 3-1	W value	21.000	55.500	0.000	-	2.500	-	-
	p-value	0.025	0.803	0.000	-	0.000	-	-
Set 3-2	W value	30.000	51.500	0.000	-	-	-	-
	p-value	0.094	0.638	6.103515625e-05	-	-	-	-
Set 4	W value	14.000	18.000	0.000	0.000	0.000	0.000	0.000
	p-value	0.359	0.652	0.003	0.003	0.003	0.003	0.003
All	W value	269.0	393.5	0.000	0.000	2.5	0.000	0.000
	p-value	0.004	0.000	0.000	0.000	0.000	0.000	0.000

- Means N/A.

5. Conclusions and Future Research

This study presented a binary version of both the SMA and SCA, with a linear relaxation method applied, called ISMA and ISCA, to address the MKP problem. A novel hybrid algorithm, HSMSCA, was developed by combining these two algorithms and incorporating a selection mechanism to optimize the solution update strategy. The linear relaxation solution was utilized to calculate the price-performance ratio of items, which not only assessed item preferences but also effectively guided the search process, thereby significantly enhancing solution-finding efficiency. An innovative algorithmic framework was proposed, which used a selection mechanism to determine whether to adopt the SMA or SCA for updating solutions, thus improving the algorithm’s adaptability and effectiveness. The effectiveness of the HSMSCA algorithm was verified on four commonly used MKP problem sets, with results showing that HSMSCA outperformed the ISMA, ISCA algorithms, and other meta-heuristic algorithms.

Future research can be pursued in two primary directions. First, the applicability of HSMSCA can be explored for other optimization problems, such as multi-objective

knapsack problems or feature selection. Second, the algorithms can be enhanced by integrating orthogonal learning, deep reinforcement learning, and adaptive learning strategies to further improve the search capabilities of the HSMSCA algorithm.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Disclosure of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Reference

1. Li, Y., Y. Tao, and F. Wang, *A compromised large-scale neighborhood search heuristic for capacitated air cargo loading planning*. European Journal of Operational Research, 2009. **199**(2): p. 553-560.
2. Engwall, M. and A. Jerbrant, *The resource allocation syndrome: the prime challenge of multi-project management?* International journal of project management, 2003. **21**(6): p. 403-409.
3. Johari, R. and J.N. Tsitsiklis, *Efficiency loss in a network resource allocation game*. Mathematics of Operations Research, 2004. **29**(3): p. 407-435.
4. Balev, S., et al., *A dynamic programming based reduction procedure for the multidimensional 0–1 knapsack problem*. European journal of operational research, 2008. **186**(1): p. 63-76.
5. Boyer, V., D. El Baz, and M. Elkihel, *Solution of multidimensional knapsack problems via cooperation of dynamic programming and branch and bound*. European Journal of Industrial Engineering, 2010. **4**(4): p. 434-449.
6. Puchinger, J., G.R. Raidl, and U. Pferschy, *The multidimensional knapsack problem: Structure and algorithms*. INFORMS Journal on Computing, 2010. **22**(2): p. 250-265.
7. Vimont, Y., S. Boussier, and M. Vasquez, *Reduced costs propagation in an efficient implicit enumeration for the 01 multidimensional knapsack problem*.

- Journal of Combinatorial Optimization, 2008. **15**(2): p. 165-178.
8. Du, D., K.-I. Ko, and X. Hu, *Design and analysis of approximation algorithms*. Vol. 62. 2012: Springer.
 9. Motwani, R. and P. Raghavan, *Randomized algorithms*. 1995: Cambridge university press.
 10. Haddar, B., et al., *A hybrid quantum particle swarm optimization for the multidimensional knapsack problem*. Engineering Applications of Artificial Intelligence, 2016. **55**: p. 1-13.
 11. Vasquez, M. and Y. Vimont, *Improved results on the 0–1 multidimensional knapsack problem*. European Journal of Operational Research, 2005. **165**(1): p. 70-81.
 12. Abdel-Basset, M., D. El-Shahat, and A.K. Sangaiah, *A modified nature inspired meta-heuristic whale optimization algorithm for solving 0–1 knapsack problem*. International Journal of Machine Learning and Cybernetics, 2019. **10**: p. 495-514.
 13. Luo, K. and Q. Zhao, *A binary grey wolf optimizer for the multidimensional knapsack problem*. Applied Soft Computing, 2019. **83**: p. 105645.
 14. Abdel-Basset, M., et al., *BSMA: A novel metaheuristic algorithm for multi-dimensional knapsack problems: Method and comprehensive analysis*. Computers & Industrial Engineering, 2021. **159**: p. 107469.
 15. Feng, Y., et al., *Hybrid Learning Moth Search Algorithm for Solving Multidimensional Knapsack Problems*. Mathematics, 2023. **11**(8): p. 1811.
 16. Li, S., et al., *Slime mould algorithm: A new method for stochastic optimization*. Future generation computer systems, 2020. **111**: p. 300-323.
 17. Houssein, E.H., et al., *Hybrid slime mould algorithm with adaptive guided differential evolution algorithm for combinatorial and global optimization problems*. Expert Systems with Applications, 2021. **174**: p. 114689.
 18. Ahmadianfar, I., et al., *Multi-strategy Slime Mould Algorithm for hydropower multi-reservoir systems optimization*. Knowledge-Based Systems, 2022. **250**: p. 109048.
 19. Alfadhli, J., et al., *FP-SMA: an adaptive, fluctuant population strategy for slime mould algorithm*. Neural Computing and Applications, 2022. **34**(13): p. 11163-11175.
 20. Rizk-Allah, R.M., A.E. Hassanien, and D. Song, *Chaos-opposition-enhanced slime mould algorithm for minimizing the cost of energy for the wind turbines on high-altitude sites*. ISA transactions, 2022. **121**: p. 191-205.
 21. Örnek, B.N., et al., *A novel version of slime mould algorithm for global optimization and real world engineering problems: Enhanced slime mould*

- algorithm*. Mathematics and Computers in Simulation, 2022. **198**: p. 253-288.
22. Ewees, A.A., et al., *Improved Slime Mould Algorithm based on Firefly Algorithm for feature selection: A case study on QSAR model*. Engineering with Computers, 2021: p. 1-15.
 23. Mirjalili, S., *SCA: a sine cosine algorithm for solving optimization problems*. Knowledge-based systems, 2016. **96**: p. 120-133.
 24. Attia, A.-F., R.A. El Sehiemy, and H.M. Hasanien, *Optimal power flow solution in power systems using a novel Sine-Cosine algorithm*. International Journal of Electrical Power & Energy Systems, 2018. **99**: p. 331-343.
 25. Gupta, S. and K. Deep, *A hybrid self-adaptive sine cosine algorithm with opposition based learning*. Expert Systems with Applications, 2019. **119**: p. 210-230.
 26. Nayak, D.R., et al., *Combining extreme learning machine with modified sine cosine algorithm for detection of pathological brain*. Computers & Electrical Engineering, 2018. **68**: p. 366-380.
 27. Li, Y., Y. Zhao, and J. Liu, *Dimension by dimension dynamic sine cosine algorithm for global optimization problems*. Applied Soft Computing, 2021. **98**: p. 106933.
 28. Chu, P.C. and J.E. Beasley, *A genetic algorithm for the multidimensional knapsack problem*. Journal of heuristics, 1998. **4**: p. 63-86.
 29. Zhang, B., et al., *An effective hybrid harmony search-based algorithm for solving multidimensional knapsack problems*. Applied Soft Computing, 2015. **29**: p. 288-297.
 30. Azad, M.A.K., A.M.A. Rocha, and E.M. Fernandes, *Improved binary artificial fish swarm algorithm for the 0-1 multidimensional knapsack problems*. Swarm and evolutionary computation, 2014. **14**: p. 66-75.
 31. Zhang, S., et al., *Grey wolf optimizer for unmanned combat aerial vehicle path planning*. Advances in Engineering Software, 2016. **99**: p. 121-136.
 32. Senju, S. and Y. Toyoda, *An approach to linear programming with 0-1 variables*. Management Science, 1968: p. B196-B207.
 33. Freville, A., *Hard 0-1 multi-knapsack test problems for size reduction methods*. Investigation Operativa, 1990.
 34. Weingartner, H.M. and D.N. Ness, *Methods for the solution of the multidimensional 0/1 knapsack problem*. Operations Research, 1967. **15**(1): p. 83-103.
 35. Shih, W., *A branch and bound method for the multiconstraint zero-one knapsack problem*. Journal of the Operational Research Society, 1979. **30**(4): p. 369-378.

36. Glover, F. and G.A. Kochenberger, *Critical event tabu search for multidimensional knapsack problems*. Meta-heuristics: theory and applications, 1996: p. 407-427.
37. Pinto, H., et al. *A binary sine-cosine algorithm applied to the knapsack problem*. in *Artificial Intelligence Methods in Intelligent Algorithms: Proceedings of 8th Computer Science On-line Conference 2019, Vol. 2* 8. 2019. Springer.

Appendix

Table A1. The Result of the Parameter Test

	SMA: $r=0.01$	SMA: $z=0.08$	SMA: $z=0.15$	SCA: $\alpha=1.5$	SCA: $\alpha=2.0$	SCA: $\alpha=2.5$	HMSCA (z)= $0.01, \alpha=1.5$	HMSCA: $z=0.01, \alpha=2.0$
GK-4	5760/5751.200/0.270	5760/5753.450/0.230	5758/5752.150/0.260	5758/5749.400/0.310	5758/5748.850/0.310	5758/5749.950/0.300	5754/5746.500/0.360	5759/5749.100/0.310
GK-9	58073/58069.650/0.030	58071/58064.250/0.040	58071/58061.500/0.050	58077/58070.450/0.030	58074/58070.950/0.030	58075/58070.150/0.030	58074/58065.850/0.040	58076/58071.650/0.030
cb3-3	121121/121053.200/0.060	121121/121057.250/0.060	121115/121034.250/0.080	121121/121088.550/0.040	121121/121093.750/0.030	121121/121091.600/0.030	121109/121038.500/0.080	121121/121078.100/0.040
cb6-2	119099/119057.150/0.160	119087/119029.600/0.180	119082/119015.450/0.200	119114/119083.950/0.140	119179/119088.950/0.130	119108/119082.950/0.140	119139/119063.150/0.160	119086/119082.200/0.140
Weish16	7289/7287.900/0.020	7289/7289.000/0.000	7289/7289.000/0.000	7289/7288.000/0.010	7289/7287.700/0.020	7289/7287.900/0.020	7289/7287.400/0.020	7289/7287.800/0.020
Weish26	9584/9581.400/0.030	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000
PB4	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/94955.050/0.220	95168/94793.400/0.390	95168/95168.000/0.000	95168/95168.000/0.000
Weing5	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000
	HMSCA: $z=0.01, \alpha=2.5$	HMSCA: $z=0.08, \alpha=1.5$	HMSCA: $z=0.08, \alpha=2.0$	HMSCA: $z=0.08, \alpha=2.5$	HMSCA: $z=0.15, \alpha=1.5$	HMSCA: $z=0.15, \alpha=2.0$	HMSCA: $z=0.15, \alpha=2.5$	
GK-4	5756/5749.650/0.300	5758/5750.450/0.290	5757/5751.100/0.280	5759/5751.650/0.270	5758/5751.450/0.270	5757/5751.350/0.270	5758/5750.650/0.280	
GK-9	58077/58070.600/0.030	58074/58066.850/0.040	58077/58068.400/0.040	58075/58068.500/0.040	58070/58066.000/0.040	58073/58066.550/0.040	58073/58066.250/0.040	
cb3-3	121109/121071.600/0.050	121106/121056.500/0.060	121093/121056.750/0.060	121121/121057.450/0.060	121109/121044.450/0.070	121121/121053.650/0.060	121109/121054.500/0.060	
cb6-2	119082/119080.450/0.140	119082/119076.350/0.140	119082/119078.000/0.140	119105/119079.950/0.140	119087/119064.600/0.150	119087/119071.200/0.150	119087/119036.200/0.180	
Weish16	7289/7288.100/0.010	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	
Weish26	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	
PB4	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	
Weing5	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	

Max/ Mean/ PDev

Table A2. The Results Obtained for Different Problem Sets.

Instance	ISMA	ISCA	HSMSCA	HLMS	BIWOA	BMMVO	BSCA	IBSMA_U1
Sento 1	7772/7772/0.000/1	7772/7762/0.134/4	7772/7772/0.000/1	-/7762.000/0.129/3	-/-/-	-/-/-	-/-/-	-/-/-
Sento 2	8722/8722/0.000/1	8722/8722/0.000/1	8722/8722/0.000/1	-/8719.000/0.034/4	-/-/-	-/-/-	-/-/-	-/-/-
HP1	3418/3416/0.061/2	3418/3414/0.123/3	3418/3417/0.020/1	-/3399.000/0.556/4	-/-/-	-/-/-	-/-/-	-/-/-
HP2	3186/3183/0.085/2	3186/3176/0.317/3	3186/3185/0.028/1	-/3158.000/0.879/4	-/-/-	-/-/-	-/-/-	-/-/-
PB1	3090/3090/0.000/1	3090/3075/0.492/3	3090/3090/0.000/1	-/3074.000/0.518/4	-/-/-	-/-/-	-/-/-	-/-/-
PB2	3186/3184/0.056/2	3186/3171/0.476/4	3186/3185/0.028/1	-/3178.000/0.251/3	-/-/-	-/-/-	-/-/-	-/-/-
PB4	95168/95168/0.000/1	95168/94955/0.224/3	95168/95168/0.000/1	-/93063.000/2.212/4	-/-/-	-/-/-	-/-/-	-/-/-
PB5	2139/2139/0.000/1	2139/2139/0.000/1	2139/2139/0.000/1	-/2125.000/0.655/4	-/-/-	-/-/-	-/-/-	-/-/-
PB6	776/774/0.213/1	765/765/1.418/4	776/774/0.213/1	-/768.000/1.031/3	-/-/-	-/-/-	-/-/-	-/-/-
PB7	1035/1035/0.000/1	1035/1035/0.000/1	1035/1035/0.000/1	-/1030.000/0.483/4	-/-/-	-/-/-	-/-/-	-/-/-
Weing1	141278/141278/0.000/1	141278/141278/0.000/1	141278/141278/0.000/1	-/141260.000/0.013/4	-/-/-	-/-/-	-/-/-	-/-/-
Weing2	130883/130883/0.000/1	130883/130763/0.092/4	130883/130883/0.000/1	-/130877.000/0.005/3	-/-/-	-/-/-	-/-/-	-/-/-
Weing3	95677/95677/0.000/1	95677/95588/0.093/3	95677/95662/0.016/2	-/94801.000/0.916/4	-/-/-	-/-/-	-/-/-	-/-/-
Weing4	119337/119337/0.000/1	119337/119337/0.000/1	119337/119337/0.000/1	-/118956.000/0.319/4	-/-/-	-/-/-	-/-/-	-/-/-
Weing5	98796/98796/0.000/1	98796/98796/0.000/1	98796/98796/0.000/1	-/98796.000/0.000/1	-/-/-	-/-/-	-/-/-	-/-/-
Weing6	130623/130623/0.000/1	130623/130331/0.224/4	130623/130623/0.000/1	-/130610.000/0.010/3	-/-/-	-/-/-	-/-/-	-/-/-
Weing7	1095445/1095385/0.005/1	1095382/1095382/0.006/2	1094757/1094757/0.063/3	-/1073783.000/1.977/4	-/-/-	-/-/-	-/-/-	-/-/-
Weing8	624319/624319/0.000/1	624319/624319/0.000/1	624319/624319/0.000/1	-/622775.000/0.247/4	-/-/-	-/-/-	-/-/-	-/-/-
Weish01	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.180/7	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.090/6	4554/-/0.000/1
Weish02	4536/-/0.000/1	4536/-/0.298/7	4536/-/0.000/1	4536/-/0.020/6	4536/-/0.000/1	4536/-/0.000/1	4536/-/0.000/1	4536/-/0.000/1
Weish03	4115/-/0.000/1	4115/-/0.077/4	4115/-/0.000/1	4115/-/0.190/5	4106/-/3.970/7	4106/-/3.050/6	4106/-/0.000/1	4115/-/0.000/1
Weish04	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.090/7	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1
Weish05	4514/-/0.000/1	4514/-/0.060/7	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1
Weish06	5557/-/0.000/1	5557/-/0.000/1	5557/-/0.000/1	5557/-/0.090/4	5557/-/0.120/5	5557/-/0.140/6	5557/-/0.140/6	5557/-/0.000/1
Weish07	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.030/7	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1
Weish08	5605/-/0.000/1	5605/-/0.000/1	5605/-/0.000/1	5605/-/0.010/5	5605/-/0.000/1	5605/-/0.030/7	5605/-/0.010/5	5605/-/0.000/1
Weish09	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1
Weish10	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.040/4	6323/-/0.310/5	6303/-/0.560/6	6303/-/0.560/6	6323/-/0.505/6
Weish11	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1
Weish12	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.070/4	6302/-/0.580/5	6301/-/0.590/6	6302/-/0.590/6	6302/-/0.584/6
Weish13	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.230/7	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1
Weish14	6954/-/0.000/1	6954/-/0.000/1	6954/-/0.000/1	6954/-/0.280/4	6923/-/0.710/7	6923/-/0.570/5	6923/-/0.660/6	6923/-/0.479/5
Weish15	7486/-/0.000/1	7486/-/0.075/4	7486/-/0.000/1	7486/-/0.080/5	7486/-/0.080/5	7486/-/0.110/7	7486/-/0.000/1	7486/-/0.000/1
Weish16	7289/-/0.000/1	7289/-/0.014/4	7289/-/0.000/1	7289/-/0.140/7	7289/-/0.010/3	7289/-/0.100/6	7289/-/0.090/5	7289/-/0.003/3
Weish17	8633/-/0.000/1	8633/-/0.073/4	8633/-/0.000/1	8633/-/0.000/1	8633/-/0.390/5	8624/-/0.960/7	8633/-/0.420/6	8633/-/0.014/4
Weish18	9580/-/0.000/1	9580/-/0.000/1	9580/-/0.000/1	9580/-/0.110/4	9560/-/0.650/6	9456/-/1.920/7	9573/-/0.620/5	9580/-/0.044/4
Weish19	7698/-/0.000/1	7698/-/0.093/5	7698/-/0.000/1	7698/-/0.010/4	7698/-/0.380/7	7698/-/0.350/6	7698/-/0.000/1	7698/-/0.000/1
Weish20	9450/-/0.000/1	9450/-/0.057/5	9450/-/0.000/1	9450/-/0.030/4	9450/-/0.230/6	9445/-/0.570/7	9450/-/0.020/3	9450/-/0.000/1
Weish21	9074/-/0.000/1	9074/-/0.000/1	9074/-/0.000/1	9074/-/0.050/5	9074/-/0.190/6	9074/-/0.640/7	9074/-/0.030/4	9074/-/0.000/1
Weish22	8947/-/0.000/1	8947/-/0.386/3	8947/-/0.000/1	8929/-/0.580/5	8909/-/0.430/4	8886/-/0.680/7	8909/-/0.630/6	8929/-/0.201/3
Weish23	8344/-/0.000/1	8344/-/0.016/3	8344/-/0.000/1	8344/-/0.850/5	8303/-/0.660/4	8250/-/1.160/7	8344/-/0.890/6	8344/-/0.000/1
Weish24	10220/-/0.000/1	10220/-/0.005/3	10220/-/0.000/1	10220/-/0.160/4	10189/-/1.230/6	10058/-/3.140/7	10215/-/0.860/5	10220/-/0.030/4
Weish25	9939/-/0.000/1	9939/-/0.113/4	9939/-/0.000/1	9939/-/0.030/3	9885/-/0.940/6	9844/-/1.630/7	9939/-/0.200/5	9939/-/0.000/1
Weish26	9584/-/0.000/1	9584/-/0.000/1	9584/-/0.000/1	9584/-/0.190/4	9575/-/0.690/5	9575/-/1.170/7	9575/-/0.920/6	9578/-/0.066/4

Weish 27	9819/-0.000/1	9819/-0.000/1	9819/-0.000/1	9819/-0.500/5	9778/-0.450/4	9589/-2.530/7	9764/-0.880/6	9781/-0.390/4
Weish28	9492/-0.000/1	9492/-0.000/1	9492/-0.000/1	9492/-0.190/4	9454/-0.490/5	9400/-1.700/7	9454/-0.780/6	9454/-0.400/5
Weish29	9410/-0.000/1	9410/-0.234/3	9410/-0.000/1	9410/-0.400/4	9369/-0.430/5	9369/-1.750/7	9369/-0.430/5	9372/-0.404/5
Weish 30	11191/-0.000/1	11191/-0.000/1	11191/-0.000/1	11187/-0.080/4	11121/-1.230/6	11025/-2.490/7	11169/-0.610/5	11191/-0.032/4
cb1-1	24381/24365.200/0.065/3	24381/24367.700/0.055/2	24381/24369.600/0.047/1	24381/24301.000/0.328/4	-/-	24192/24050.000/1.358/5	-/-	-/-
cb1-2	24274/24265.200/0.036/4	24274/24271.800/0.009/2	24274/24267.400/0.027/3	24274/24231.000/0.177/5	-/-	24274/24274.000/0.000/1	-/-	-/-
cb1-3	23538/23527.450/0.100/1	23534/23523.550/0.117/3	23538/23524.850/0.111/2	23551/23521.000/0.127/4	-/-	23538/23520.000/0.132/5	-/-	-/-
cb1-4	23511/23489.000/0.191/1	23534/23485.900/0.204/3	23527/23486.800/0.201/2	23503/23420.000/0.484/4	-/-	23288/23120.000/1.759/5	-/-	-/-
cb1-5	23966/23958.450/0.136/2	23991/23950.350/0.169/3	23991/23959.000/0.133/1	23966/23937.000/0.225/4	-/-	23947/23900.000/0.379/5	-/-	-/-
cb 2-1	59312/59222.250/0.151/3	59243/59241.150/0.119/1	59243/59223.850/0.149/2	59063/58862.000/0.759/4	-/-	58473/58240.000/1.807/5	-/-	-/-
cb 2-2	61454/61410.000/0.101/2	61450/61374.600/0.158/3	61468/61428.350/0.071/1	61295/61051.000/0.685/4	-/-	60692/60390.000/1.760/5	-/-	-/-
cb 2-3	62065/62027.850/0.164/2	62057/61970.450/0.257/3	62057/62031.800/0.158/1	61767/61552.000/0.930/4	-/-	61702/61330.000/1.288/5	-/-	-/-
cb 2-4	59427/59394.050/0.116/3	59429/59407.400/0.094/1	59425/59398.850/0.108/2	59140/58922.000/0.910/4	-/-	58441/58300.000/1.956/5	-/-	-/-
cb 2-5	58950/58930.450/0.035/3	58950/58945.450/0.009/1	58950/58937.650/0.023/2	58605/58390.000/0.952/4	-/-	58082/58300.000/1.104/5	-/-	-/-
cb3-1	120130/120055.850/0.077/2	120116/120083.400/0.054/1	120090/120053.600/0.079/3	119101/118457.000/1.407/5	-/-	119978/119900.000/0.206/4	-/-	-/-
cb3-2	117823/117768.000/0.094/3	117824/117811.750/0.057/1	117814/117779.600/0.084/2	116227/115704.000/1.845/4	-/-	115634/115400.000/2.103/5	-/-	-/-
cb3-3	121121/121057.250/0.061/3	121121/121088.550/0.035/1	121121/121057.450/0.061/2	119990/119468.000/1.373/4	-/-	119156/118900.000/1.842/5	-/-	-/-
cb3-4	120785/120710.800/0.077/3	120767/120731.200/0.060/2	120782/120731.250/0.060/1	119015/118386.000/2.002/5	-/-	119124/118900.000/1.576/4	-/-	-/-
cb3-5	122288/122256.200/0.051/3	122319/122264.200/0.045/1	122280/122257.400/0.050/2	120918/120278.000/1.669/5	-/-	121141/120800.000/1.242/4	-/-	-/-
cb4-1	23064/23048.700/0.066/2	23057/23046.300/0.077/3	23064/23052.150/0.051/1	23055/22914.000/0.650/4	-/-	-/-	-/-	-/-
cb4-2	22801/22762.750/0.168/1	22801/22750.700/0.221/3	22801/22753.100/0.210/2	22743/22629.000/0.754/4	-/-	-/-	-/-	-/-
cb4-3	22131/22097.000/0.154/2	22131/22075.050/0.253/3	22131/22097.050/0.153/1	22131/21908.000/1.008/4	-/-	-/-	-/-	-/-
cb4-4	22772/22726.700/0.199/1	22772/22708.600/0.278/3	22772/22724.900/0.207/2	22717/22528.000/1.071/4	-/-	-/-	-/-	-/-
cb4-5	22751/22635.050/0.510/1	22751/22568.200/0.803/4	22751/22614.150/0.602/2	22751/22603.000/0.651/3	-/-	-/-	-/-	-/-
cb5-1	59181/59057.150/0.219/3	59127/59098.150/0.150/1	59127/59072.400/0.194/2	58903/58477.000/1.200/4	-/-	-/-	-/-	-/-
cb5-2	58651/58559.350/0.377/3	58651/58605.950/0.298/1	58605/58570.550/0.358/2	58346/58098.000/1.162/4	-/-	-/-	-/-	-/-
cb5-3	58034/57944.400/0.263/3	58056/57986.650/0.190/1	58055/57971.900/0.215/2	57674/57417.000/1.170/4	-/-	-/-	-/-	-/-
cb5-4	60888/60831.300/0.277/2	60926/60797.200/0.332/3	60977/60867.500/0.217/1	60505/60282.000/1.177/4	-/-	-/-	-/-	-/-
cb5-5	58060/57964.700/0.219/3	58019/57964.800/0.219/2	58060/57975.900/0.200/1	57468/57220.000/1.501/4	-/-	-/-	-/-	-/-
bb6-1	117639/117566.450/0.216/2	117667/117616.000/0.174/1	117659/117562.750/0.219/3	116015/115379.000/2.073/4	-/-	-/-	-/-	-/-
cb6-2	119087/119029.600/0.184/3	119114/119083.950/0.138/1	119105/119079.950/0.142/2	117778/117102.000/1.800/4	-/-	-/-	-/-	-/-
cb6-3	119085/119013.000/0.169/3	119085/119055.950/0.133/1	119085/119020.850/0.163/2	117345/116842.000/1.991/4	-/-	-/-	-/-	-/-
cb6-4	118706/118632.750/0.165/3	118727/118706.600/0.103/1	118727/118638.800/0.160/2	117281/116446.000/2.005/4	-/-	-/-	-/-	-/-
cb6-5	116423/116302.100/0.196/3	116458/116382.200/0.127/1	116404/116309.200/0.189/2	114909/114183.000/2.014/4	-/-	-/-	-/-	-/-
GK-1	3766/3760.050/0.158/1	3759/3757.050/0.238/3	3766/3759.450/0.174/2	3752/3742.000/0.637/5	3743/3736.000/0.797/6	3698/3678.000/2.337/8	3725/3705.000/1.620/7	3756/3745.200/0.552/4
GK-2	3952/3949.500/0.215/3	3952/3949.650/0.211/2	3952/3950.700/0.184/1	3948/3938.000/0.505/4	3949/3931.000/0.682/6	3885/3871.000/2.198/8	3913/3897.000/1.541/7	3944/3935.800/0.561/5
GK-3	5645/5637.700/0.324/3	5644/5639.550/0.291/2	5646/5640.000/0.283/1	5610/5596.000/1.061/5	5613/5596.000/1.061/5	5561/5519.000/2.422/8	5563/5543.000/1.998/7	5617/5610.300/0.808/4
GK-4	5760/5753.450/0.235/1	5758/5749.400/0.305/3	5759/5751.650/0.266/2	5733/5710.000/0.988/5	5712/5701.000/1.144/6	5651/5638.000/2.237/8	5678/5664.000/1.786/7	5732/5715.200/0.898/4
GK-5	7555/7550.350/0.128/3	7555/7551.900/0.107/2	7555/7552.100/0.104/1	7502/7474.000/1.138/6	7499/7485.000/0.992/5	7365/7353.000/2.738/8	7411/7391.000/2.235/7	7528/7507.600/0.693/4
GK-6	7665/7658.900/0.236/1	7665/7656.300/0.270/3	7663/7656.650/0.265/2	7611/7592.000/1.107/6	7607/7597.000/1.042/5	7522/7505.000/2.240/8	7551/7532.000/1.889/7	7649/7622.900/0.705/4
GK-7	19210/19200.050/0.109/3	19213/19209.400/0.060/1	19213/19205.200/0.082/2	19151/19067.000/0.801/6	19110/19102.000/0.619/5	18738/18668.000/2.877/8	18783/18734.000/2.534/7	19173/19146.100/0.390/4
GK-8	18780/18770.000/0.191/2	18776/18772.250/0.179/1	18782/18762.950/0.229/3	18642/18594.000/1.127/6	18641/18627.000/0.952/5	18385/18361.000/2.366/8	18462/18428.000/2.010/7	18705/18681.400/0.663/4
GK-9	58071/58064.250/0.043/3	58077/58070.450/0.032/1	58075/58068.500/0.035/2	57886/57547.000/0.933/6	57868/57843.000/0.423/5	56746/56619.000/2.531/8	56932/56719.000/2.358/7	57915/57889.600/0.343/4

Max/Mean/Pdev/Rank

-: Non Provided.